form of TDD. In an outside-in approach, the description of the low-level technical behavior of the application flows naturally from discussing, thinking about, automating, and implementing the high-level acceptance criteria (see figure 10.2).

There are many benefits of outside-in development, but the principle motivations are summarized here:

- Outside-in code focuses on business value.
- Outside-in code encourages well-designed, easy to understand code.
- Outside-in code avoids waste.

OUTSIDE-IN CODE IS FOCUSED ON BUSINESS VALUE

Outside-in code starts with the expected outcomes expressed in business terms, so any code you write can be traced back to some form of business value. The acceptance criteria help keep you focused on where the value is coming from and what you're trying to achieve.

OUTSIDE-IN CODE IS WELL-DESIGNED AND EASIER TO UNDERSTAND

When you write code from the outside in, you write an example of how a method should be used before implementing the method. This makes you think about how the code will be used by other developers, which tends to lead to cleaner API design. These examples also illustrate and document your classes and APIs, making the code much easier to understand for any new developers (or your future self).

OUTSIDE-IN CODE AVOIDS WASTE

If you strictly apply this outside-in approach, any code you write is written with the goal of making an acceptance test pass. If this isn't the case, then either some necessary aspect of the system hasn't been specified in the acceptance criteria, or you're writing code that won't be used in production. BDD won't guarantee that no unnecessary code gets written, but it will highlight potential situations where this might be the case.

Let's see what outside-in development looks like in action with a practical example.

10.2.2 Start with a high-level acceptance criterion

When practicing outside-in development, you usually start with the acceptance criteria of the feature or story that you're currently building. Suppose, for example, that you need to add support for member status levels to your Frequent Flyer application. Flying High Frequent Flyers have different status levels (Bronze, Silver, Gold, and Platinum) depending on how often they fly. Members earn status points with each flight, and when they've earned enough points over the period of a year, they move up to the next level. To encourage travellers to fly with Flying High, members with higher status levels get extra privileges, such as access to lounges, fast lanes, and so forth. At the end of the year, members retain their status level, but their status points are reset to zero.

When implementing a feature, BDD practitioners like to begin with the high-level acceptance criteria, often automating these acceptance criteria using tools like Cucumber, JBehave, and SpecFlow. Using Java and Cucumber, the acceptance criteria might look like this: